# Mobile Edge Inference Benchmark: Hosted vs On-Device Vision Pipelines

Oussama Touahri
*University of Ottawa*
Ottawa, ON, Canada
Email: otoua046@uottawa.ca

***Abstract*—This technical report evaluates the performance trade-offs between hosted and on-device computer vision inference pipelines on modern smartphones. The benchmark is based on a fixed-image perception pipeline that detects a chessboard and its pieces, and compares the latency and stability of remote API inference against fully local mobile inference. Experimental results show that on-device inference provides an order-of-magnitude reduction in perception latency while also improving execution consistency. These findings support the use of edge-based inference for real-time mobile vision applications.**

***Index Terms*—mobile machine learning, edge inference, on-device inference, computer vision, benchmarking, latency analysis**

## I. Introduction

The rapid growth of mobile computer vision applications has shifted machine learning deployment from purely cloud-based architectures toward distributed paradigms that include edge and on-device inference. While cloud platforms provide access to high-performance compute resources, they introduce unavoidable round-trip latency and non-deterministic execution delays due to network variability [1].

In contrast, modern smartphones integrate heterogeneous computing architectures with dedicated Neural Processing Units (NPUs), enabling low-latency and deterministic inference directly on-device [2]. This transition is particularly important for real-time applications where responsiveness and stability are critical.

This work benchmarks hosted and on-device inference pipelines using a controlled mobile vision task. The objective is to quantify latency differences, analyze system-level bottlenecks, and evaluate the suitability of each approach for real-time perception workloads.

## II. Experimental Platform

All experiments were conducted on an Apple iPhone Air equipped with the Apple A19 Pro system-on-chip, running iOS 26.3.1. The device integrates CPU, GPU, and a dedicated Neural Engine (NPU) for on-device machine learning inference.

Modern Apple SoCs are designed to support efficient execution of neural network workloads through hardware-accelerated frameworks such as CoreML, enabling low-latency inference directly on-device without reliance on external compute resources [3].

The reported measurements correspond to end-to-end perception latency observed at the application level, rather than isolated hardware-level performance metrics.

All benchmarks were executed under normal operating conditions without external cooling, reflecting realistic usage scenarios for mobile applications. Steady-state performance was measured after excluding warm-up runs.

This hardware configuration enables consistent low-latency inference, as reflected in the tightly bounded execution times observed in the local pipeline.

## III. System Overview

The benchmark is based on a mobile computer vision application that processes images of chessboards. The system implements a perception pipeline consisting of two sequential stages:

- **Board Detection:** localization and cropping of the chessboard region.
- **Piece Detection:** classification and localization of individual pieces.

This design reflects a common structure in vision systems, where spatial localization is followed by object-level classification. Such pipelines are representative of real-time mobile vision workloads including augmented reality and scene understanding [2].

The benchmark isolates this perception pipeline by excluding downstream processing stages, ensuring that measurements reflect only the core inference workload.

These two stages form the core perception workload used in both hosted and on-device configurations. The benchmark excludes downstream tasks such as board-state reconstruction and chess engine evaluation in order to isolate the perception pipeline itself.

## IV. Methodology

### A. Pipeline Configurations

Two inference modes are evaluated:

- **Hosted:** both board detection and piece detection are performed using remote API calls.
- **Local:** both stages are executed directly on-device using mobile inference models.

The application uses a unified architecture with a shared user interface and shared downstream logic. Only the inference backend differs between modes.

### B. Benchmark Setup

All experiments use a fixed input image to eliminate variability from camera capture conditions. Benchmarking is automated through an internal runner that executes the same perception pipeline repeatedly and logs results to a CSV file.

Warm-up runs are excluded to remove initialization overhead. Only successful steady-state runs are included, resulting in a balanced dataset of 300 executions (150 hosted, 150 local).

### C. Metrics

The primary metric is *perception latency*, defined as:

$$\text{Perception Latency} = \text{board\_ms} + \text{piece\_ms} \tag{1}$$

This isolates neural network inference time and excludes downstream processing such as FEN generation and engine evaluation.

## V. Results

### A. Perception Latency Comparison

TABLE I
PERCEPTION LATENCY COMPARISON

| Mode | Mean | Median | Std | Min | Max |
|---|---|---|---|---|---|
| Hosted | 2478.20 | 2244.50 | 1204.60 | 747.01 | 9794.47 |
| Local | 63.66 | 63.74 | 0.50 | 62.53 | 65.38 |

The local pipeline achieves a mean speedup of approximately $38.9\times$ and a median speedup of $35.2\times$ over the hosted pipeline.

### B. Latency Breakdown

TABLE II
LATENCY BREAKDOWN BY STAGE

| Mode | Board Detection (ms) | Piece Detection (ms) |
|---|---|---|
| Hosted | 820.50 | 1657.70 |
| Local | 44.75 | 18.91 |

### C. Bottleneck Analysis

TABLE III
BOTTLENECK ANALYSIS

| Mode | Board Ratio | Piece Ratio | Dominant Stage |
|---|---|---|---|
| Hosted | 33.1% | 66.9% | Piece Detection |
| Local | 70.3% | 29.7% | Board Detection |

## VI. Discussion

The results demonstrate a clear advantage for on-device inference. The local pipeline reduces perception latency from approximately 2.48 seconds to 63.66 ms, corresponding to a $\sim 39\times$ speedup. This improvement is primarily due to the elimination of network overhead, which dominates hosted execution time [1].

In addition to speed, execution stability differs significantly. The hosted pipeline exhibits high variability (standard deviation of 1204.60 ms), with latency spikes approaching 10 seconds. In contrast, the local pipeline remains tightly bounded (standard deviation of 0.50 ms), enabling consistent real-time performance.

Within the local pipeline, board detection is identified as the primary bottleneck, accounting for approximately 70.3% of total perception latency. The board detection stage (44.75 ms) is approximately 2.36 times slower than the piece detection stage (18.91 ms), despite operating on a structurally simpler detection objective.

This behavior can be attributed to differences in model architecture and deployment efficiency. The board detector uses an older Roboflow 3.0 object detection model, while the piece detector uses a more recent YOLOv11 architecture. Modern models such as YOLOv11 are designed with improved deployment compatibility and more efficient inference pipelines, particularly for mobile execution.

Legacy models such as the Roboflow 3.0 detector are less optimized for mobile deployment and may rely on additional post-processing steps (e.g., Non-Maximum Suppression) and less efficient computation graphs. These factors can introduce execution overhead on-device, particularly when operations are not fully accelerated by the Neural Engine.

In contrast, newer architectures such as YOLOv11 are designed with deployment efficiency in mind, enabling more streamlined inference pipelines and improved compatibility with mobile hardware acceleration frameworks. This results in significantly lower execution latency despite increased model complexity.

## VII. Limitations

The hosted inference results include network round-trip latency, which depends on geographic location and server proximity. While improved proximity may reduce latency, the presence of large outliers suggests that network variability remains a dominant factor [4].

Additionally, this benchmark focuses exclusively on the perception pipeline and does not include downstream processing such as FEN generation or engine evaluation.

## VIII. Conclusion

This benchmark demonstrates that on-device inference provides substantial advantages over hosted inference for mobile perception tasks. The local pipeline achieves significantly lower latency and near-deterministic execution, making it suitable for real-time applications, while hosted inference remains constrained by network variability and high latency.

The analysis further shows that once network dependency is removed, performance bottlenecks shift to local model execution. In this system, board detection dominates latency due to differences in model architecture and deployment efficiency.

These results highlight that model selection for mobile deployment should prioritize hardware compatibility and execution efficiency rather than nominal model complexity alone.

Consequently, future optimization efforts should focus on replacing legacy models with architectures specifically designed for efficient on-device inference.

## REFERENCES


[1] Cloud Is Closer Than It Appears: Revisiting the Tradeoffs of Distributed Real-Time Inference, NSF. [Online]. Available: https://par.nsf.gov/servlets/purl/10627004

[2] Machine Learning Model Deployment in Mobile-based Systems: State of the Art and Trends, ResearchGate, 2026. [Online]. Available: https://www.researchgate.net/publication/397512579_Machine_Learning_Model_Deployment_in_Mobile-based_Systems_State_of_the_Art_and_Trends

[3] What are the Top AI Chips Expected to Launch in 2025, Engineers Garage. [Online]. Available: https://www.engineersgarage.com/what-top-ai-chips-are-expected-to-launch-in-2025/

[4] Understanding Serverless Inference in Mobile-Edge Networks, IEEE Computer Society. [Online]. Available: https://www.computer.org/csdl/journal/cc/2025/01/10814988/22Wq6fxilzi


## APPENDIX A: BENCHMARK LOGGING FORMAT

The benchmark results were recorded in a CSV file with the following schema:

- `timestamp`: ISO 8601 timestamp of the run
- `pipeline_mode`: hosted or local
- `flow_type`: photo
- `board_ms`: board detection latency
- `piece_ms`: piece detection latency
- `total_ms`: total perception latency
- `success`: execution success flag

The logging system was integrated directly into the application and records one row per completed inference execution.

## APPENDIX B: CODE AND DATASET AVAILABILITY

The implementation of the mobile inference benchmarking system is available at:

- GitHub repository: https://github.com/otoua046/mobile-edge-inference-benchmark/tree/seg4180-inference-study
- Benchmark dataset: https://docs.google.com/spreadsheets/d/1UmVi-1I9MJ_NdV5iYAzWhkVoq7-rVo7BrWtiKsFWnzM/edit?usp=sharing